

ОСНОВНЫЕ ОЦЕНКИ СЛОЖНОСТИ АЛГОРИТМОВ

06

бинарное дерево поиска •
splay-дерево • случайные числа

Нестеров Р.А., PhD, доцент
департамента программной инженерии

июль 2025

30	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31	1	2	3

план лекции

01

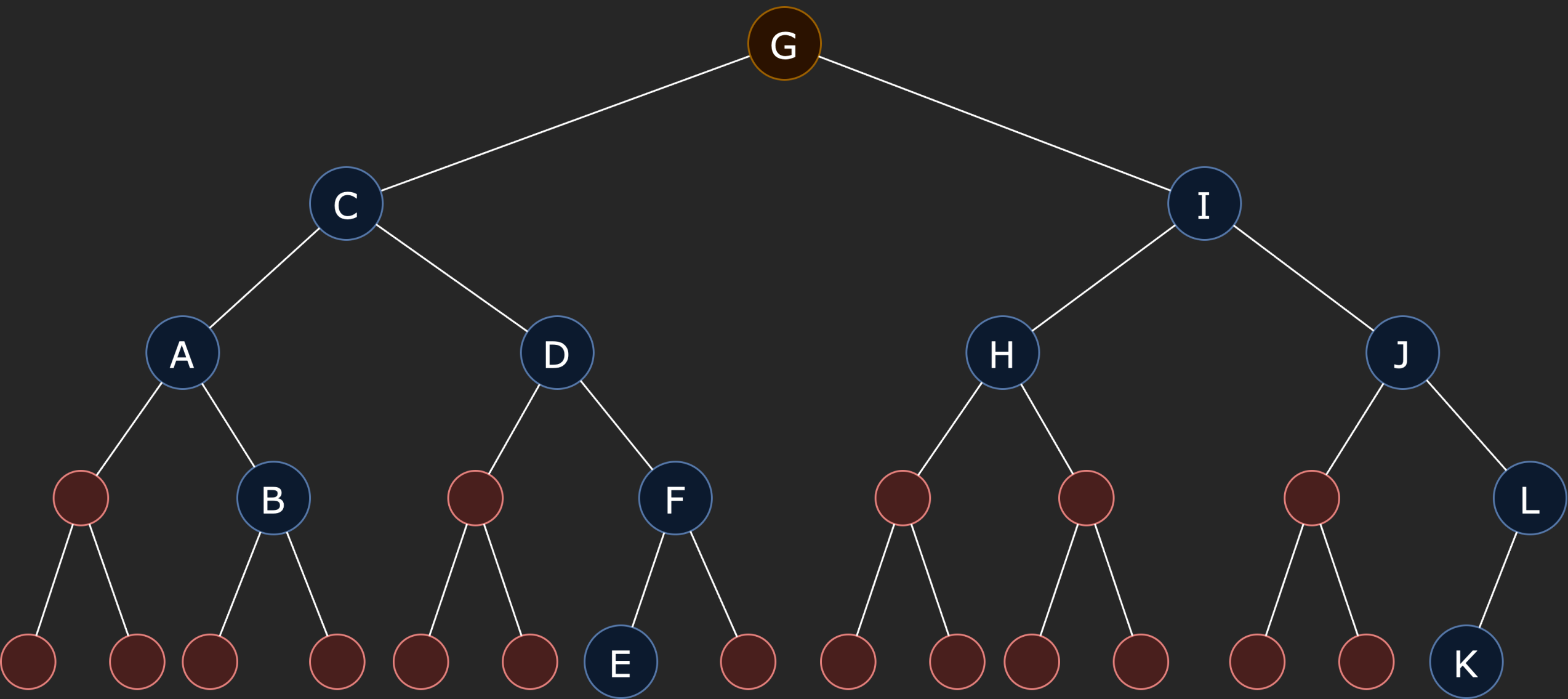
splay-дерево:
перемещение
в корень

02

splay-дерево:
повороты **zig**,
zig-zig, **zig-zag**

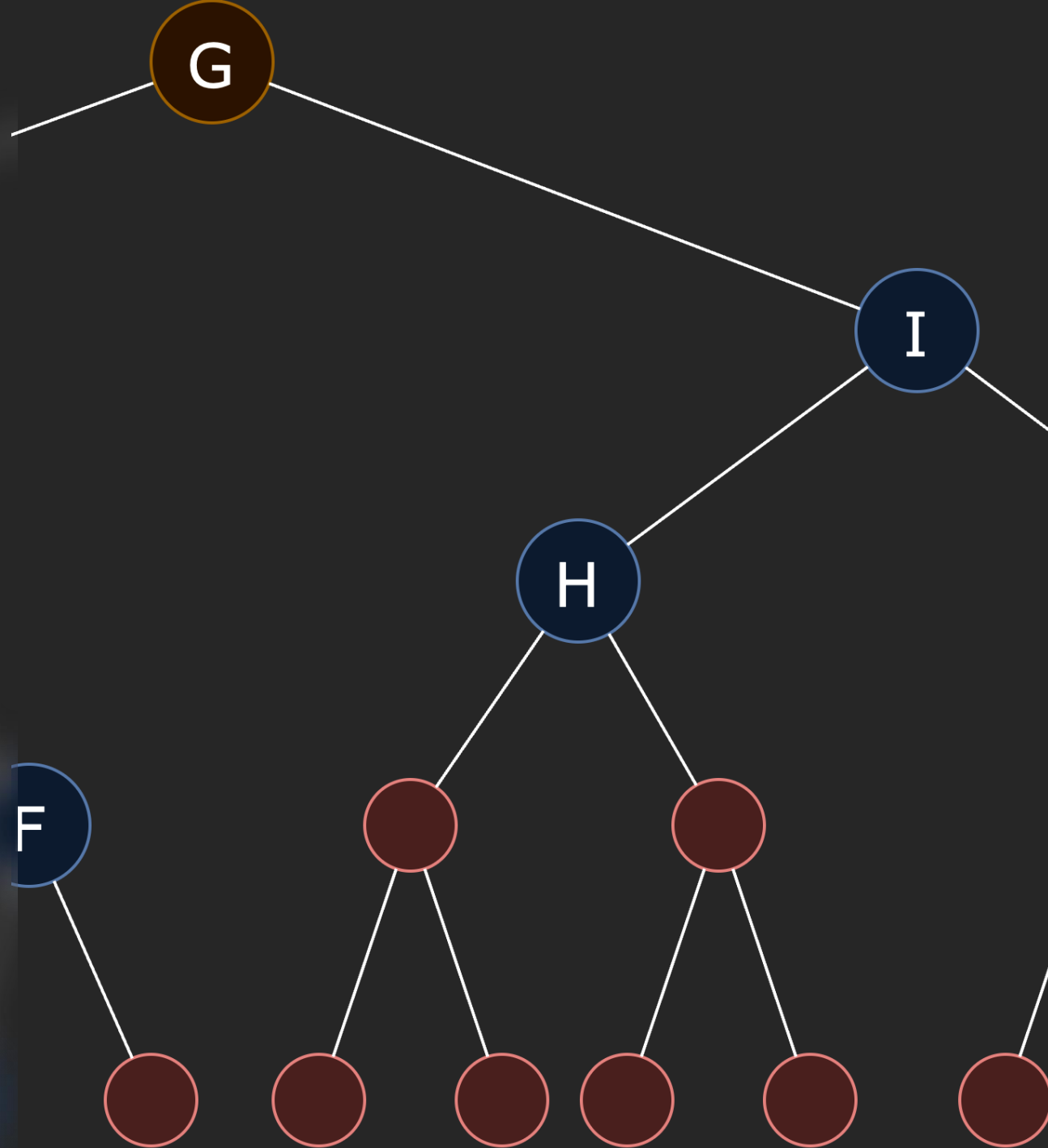
03

случайные числа:
подходы к
генерации



время выполнения операций
определяется высотой — $\Theta(h)$

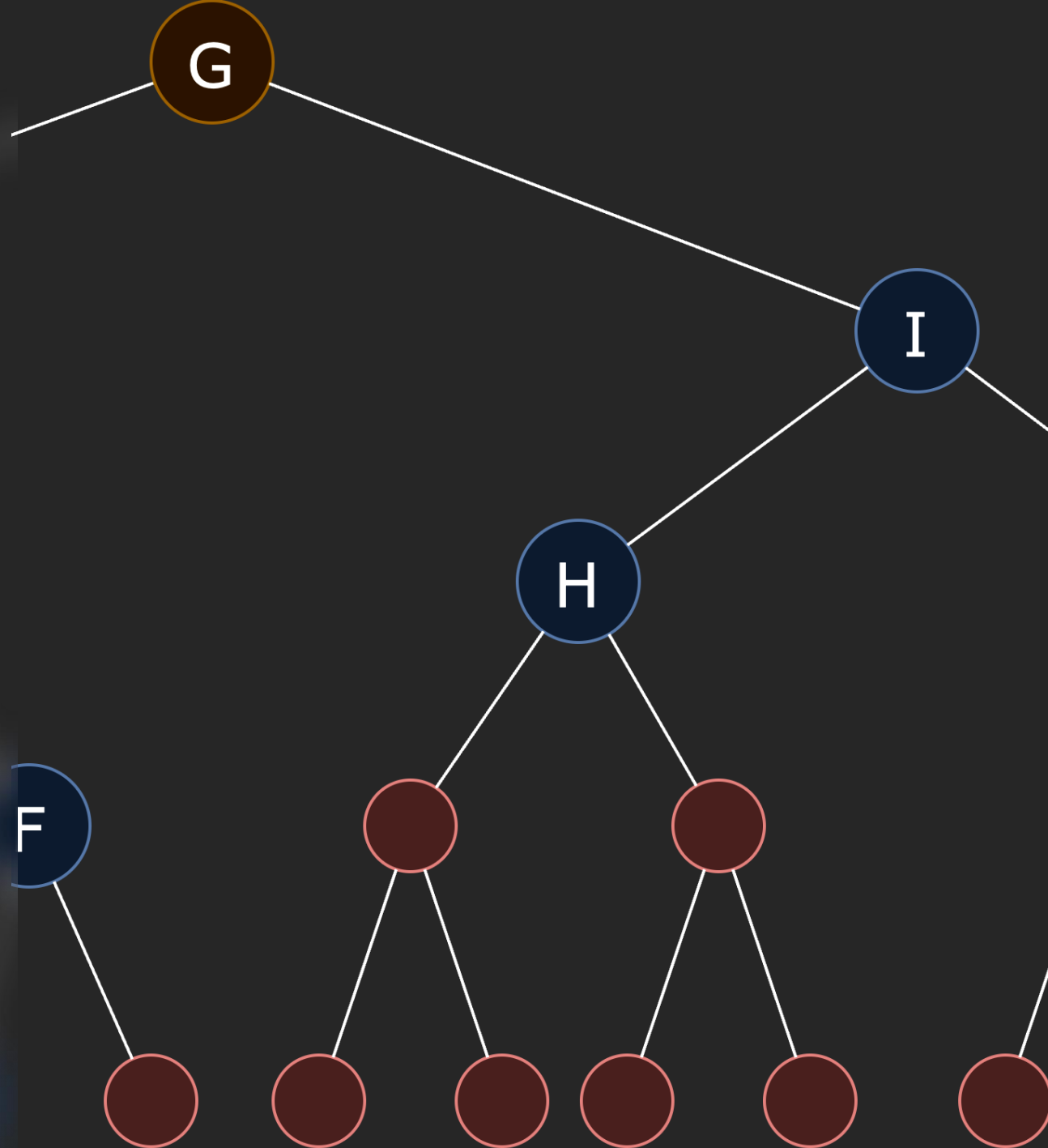
умеем ограничивать высоту —
RBT, AVL, B-дерево,
BB[α], Treap, Scapegoat,
AA-дерево, ...



splay—деревья намного
проще в реализации

не нужна дополнительная
информация о способе
ограничения высоты

часто используемые ключи
хранятся **ближе к корню**



временные затраты на выполнение операций
в splay-дереве составляют
амортизированную $\Theta(\log n)$

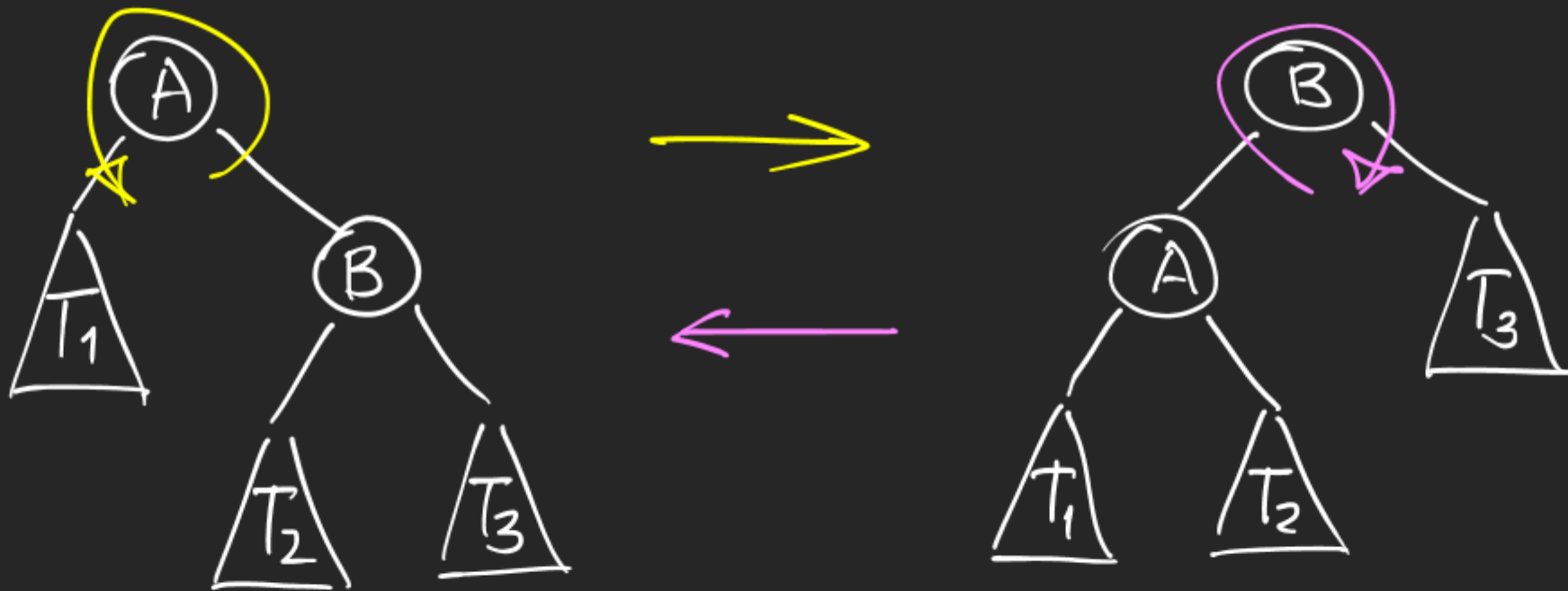
Sleator & Tarjan 1985

как перемещать значения в корень?

наивный подход выполнять [одинарные] повороты с целью перемещения искомого ключа в корневую вершину

как перемещать значения в корень?

наивный подход выполнять [одинарные] повороты с целью перемещения искомого ключа в корневую вершину

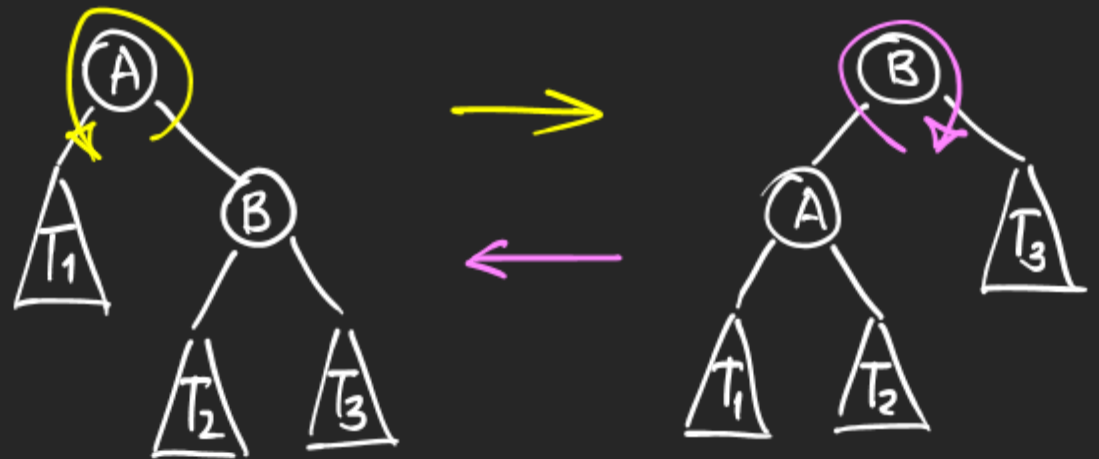


как перемещать значения в корень?

наивный подход выполнять [**одинарные**] повороты с целью перемещения искомого ключа в корневую вершину

ОДНАКО

в случае с вырожденным деревом простое применение поворотов для перестроения приведет к тому, что оценка ухудшится до $\Omega(n)$



SETUP для splay-дерева

N

общее количество ключей

их всегда можно перенумеровать,
поэтому обозначим **{1, 2, 3, ..., n}**

M

общее количество выполняемых операций

складывается из того, сколько выполняется
поисков, вставок и удалений

SETUP для splay-дерева

N

общее количество ключей

их всегда можно перенумеровать,
поэтому обозначим $\{1, 2, 3, \dots, n\}$

M

общее количество выполняемых операций

складывается из того, сколько выполняется
поисков, вставок и удалений

FIND(K)

а есть ли в splay-дереве ключ
со значением **K**?

INSERT(K)

стандартная операция
вставки нового значения
на правильное место

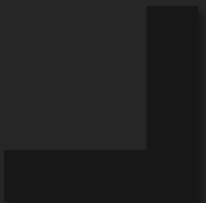
ERASE(K)

стандартная операция
удаления из дерева поиска



обрабатываемое значение **должно**
в итоге оказаться в корне splay-дерева...

почему нам совершенно не подходят
стандартные одинарные повороты?





right Rotate x 5





ТАКИМ ЖЕ ОБРАЗОМ

высота дерева не уменьшится
при сдвиге
вершин 2, ..., 6 в корень

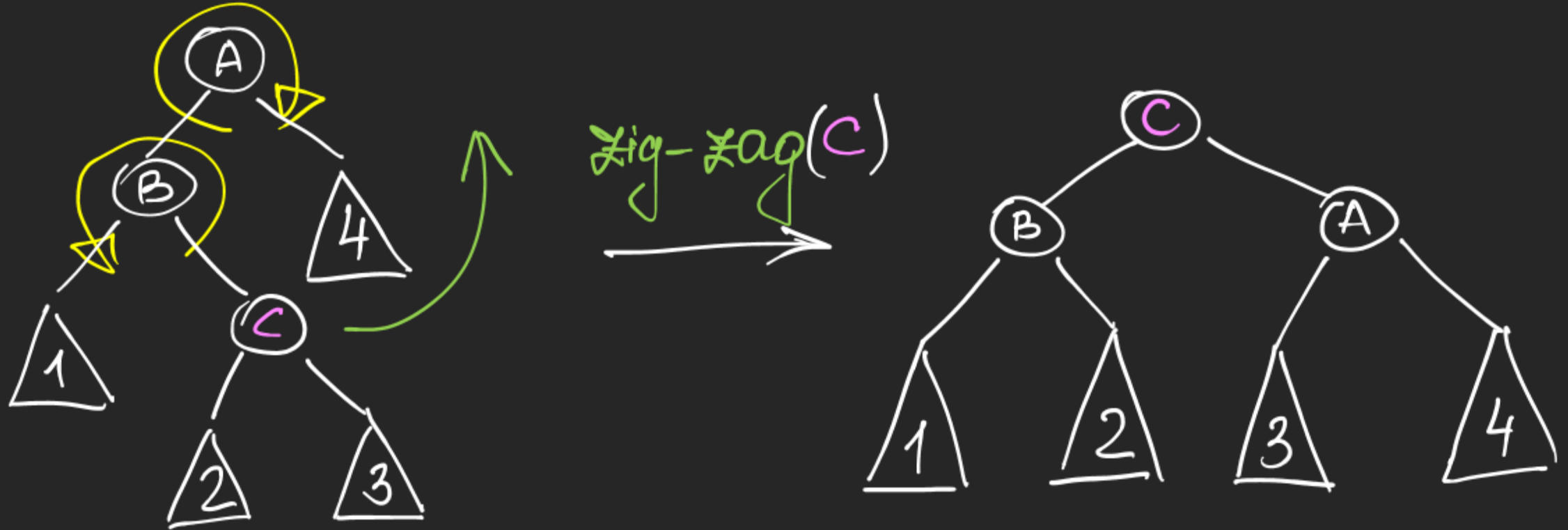


ТАКИМ ЖЕ ОБРАЗОМ

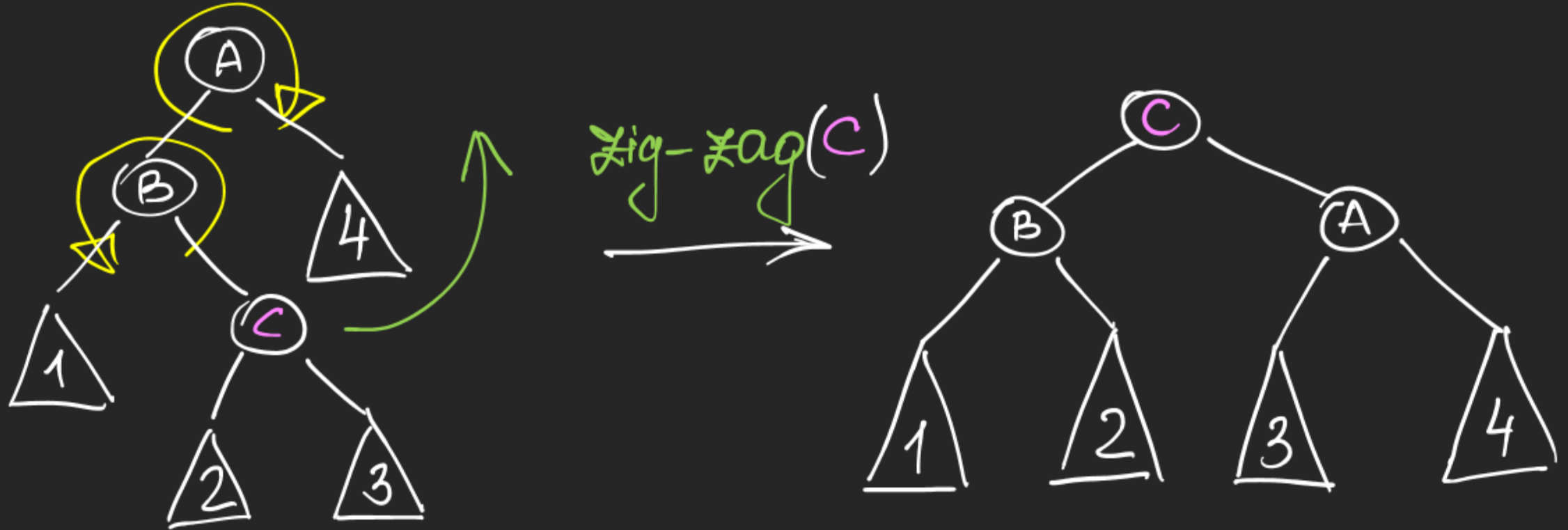
высота дерева не уменьшится
при сдвиге
вершин 2, ..., 6 в корень

НУЖНЫ ПОВОРОТЫ

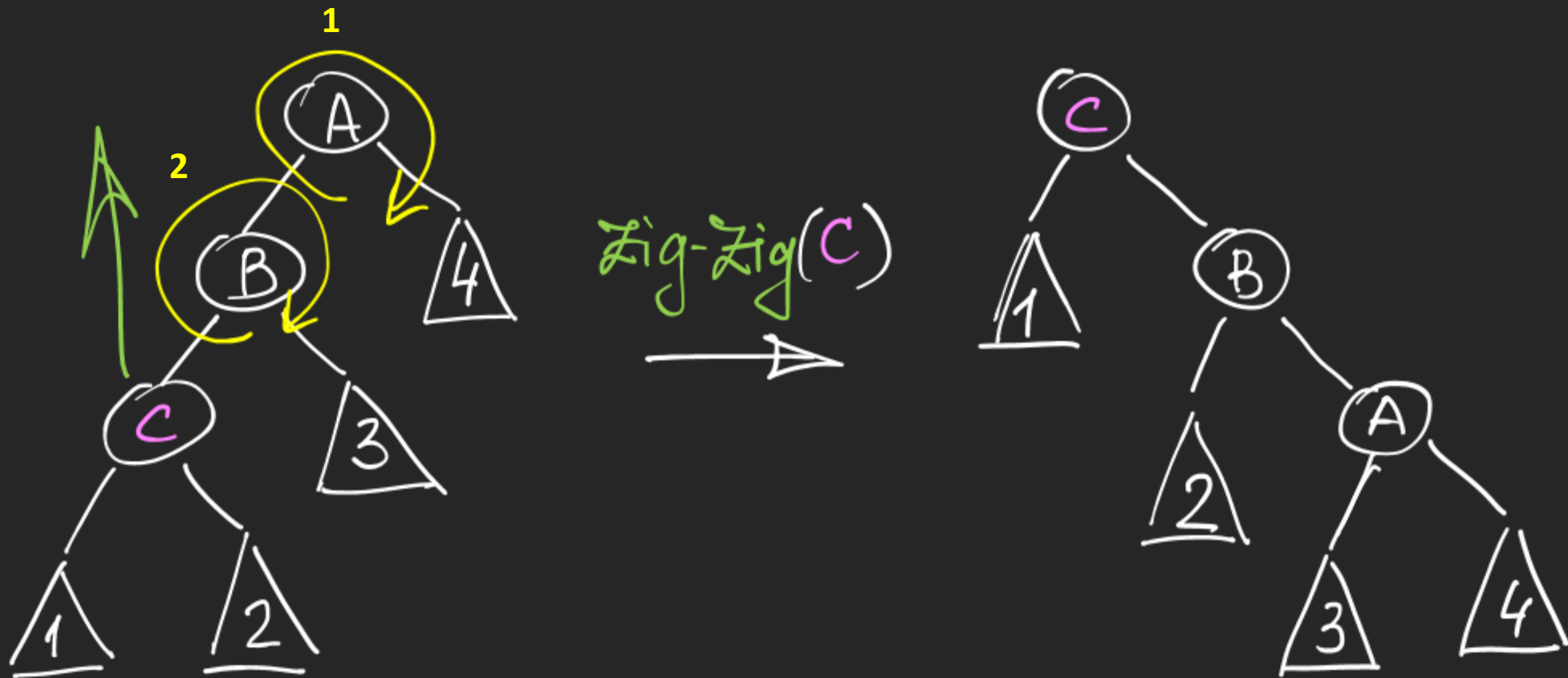
которые оптимизируют
высоту дерева на лету



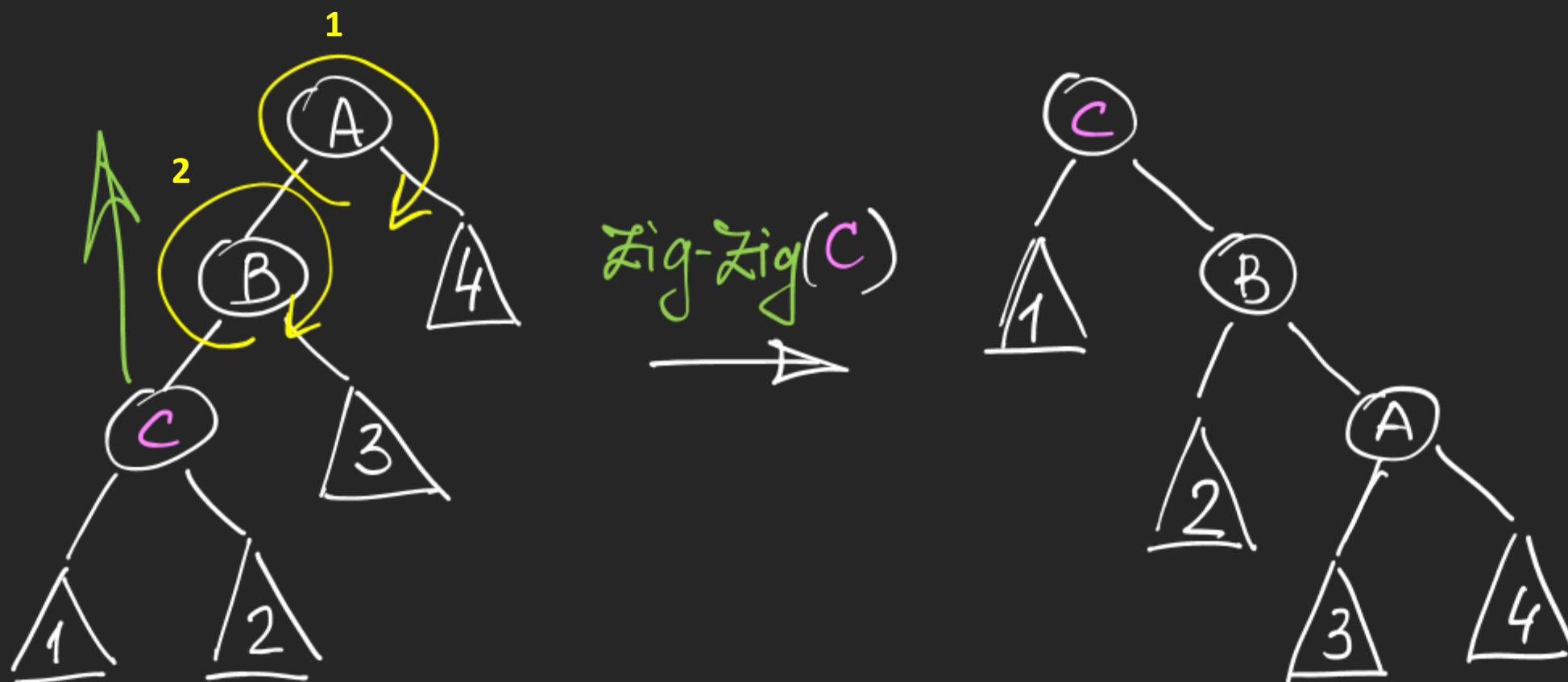
если вершина является правым потомком своего родителя, а родитель — левым потомком своего родителя, то выполняем двойной поворот относительно родителя родителя



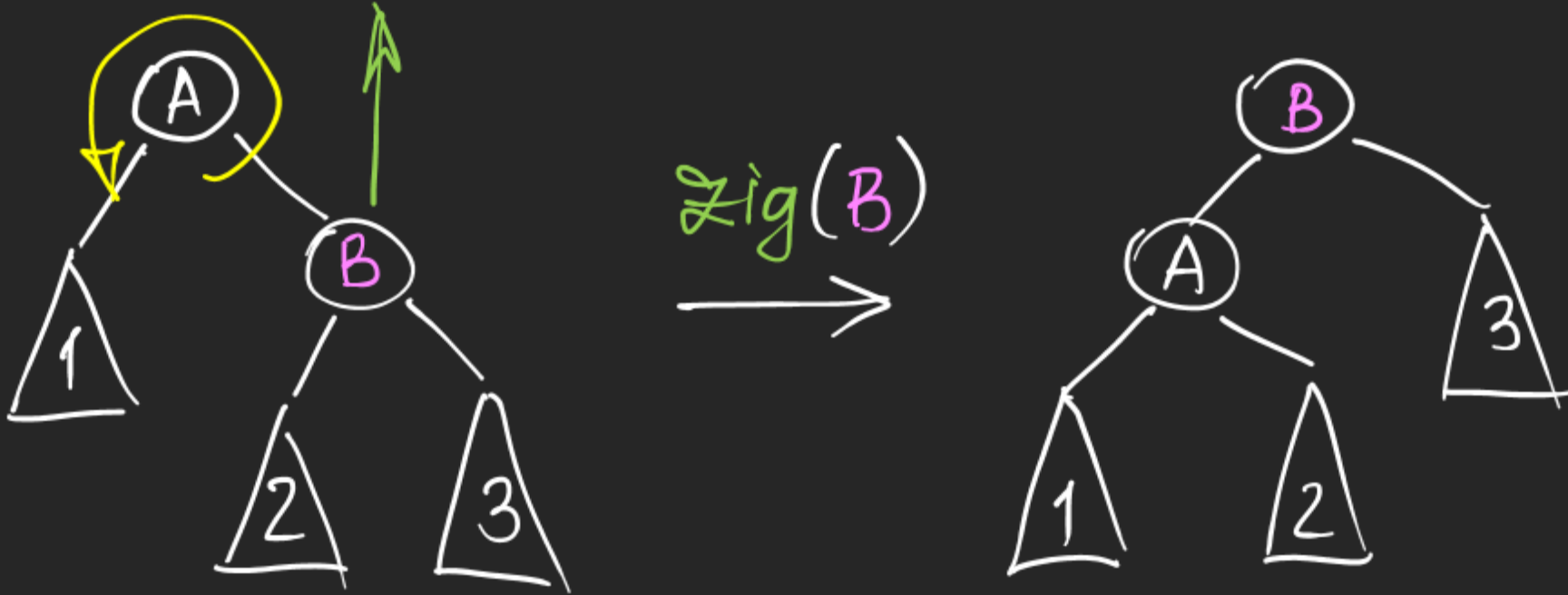
если вершина является правым **[левым]** потомком своего родителя, а родитель — левым **[правым]** потомком своего родителя, то выполняем двойной поворот относительно родителя родителя



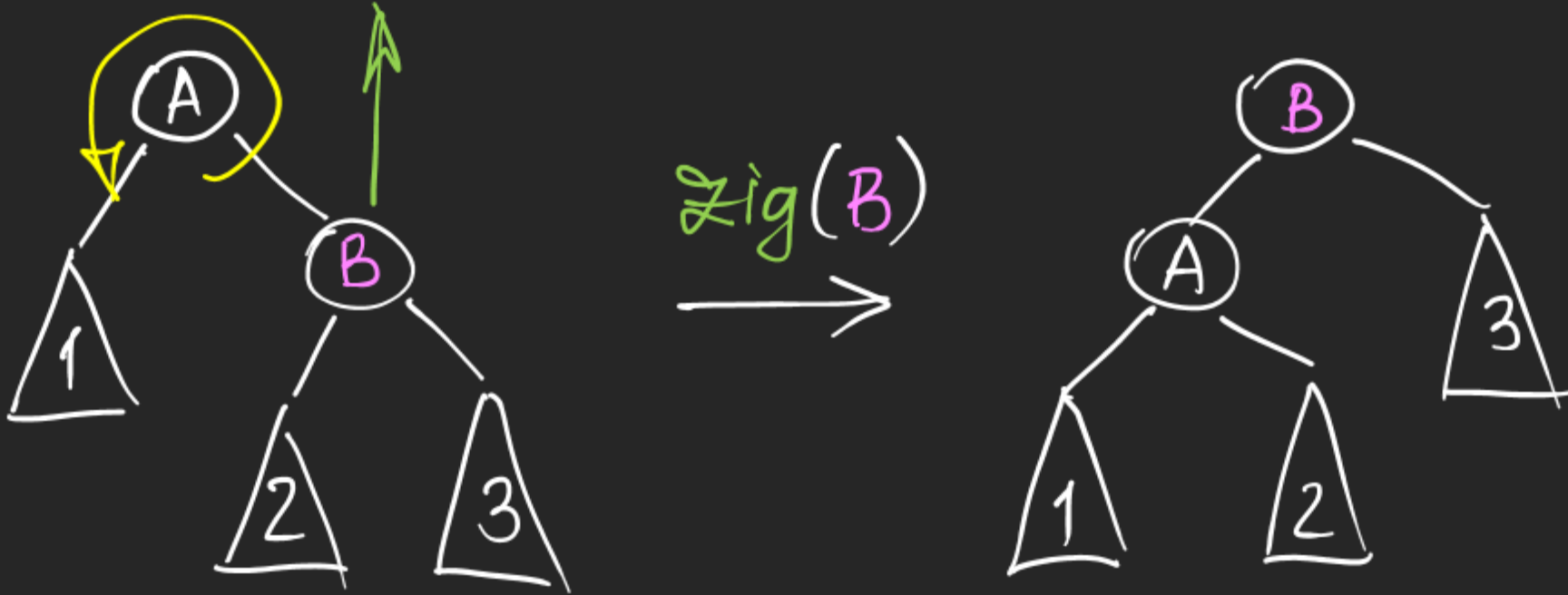
если вершина является левым потомком своего родителя,
а родитель — левым потомком своего родителя, то выполняем
два поворота относительно родителя родителя



если вершина является левым **правым** потомком своего родителя, а родитель — левым **правым** потомком своего родителя, то выполняем два поворота относительно родителя родителя



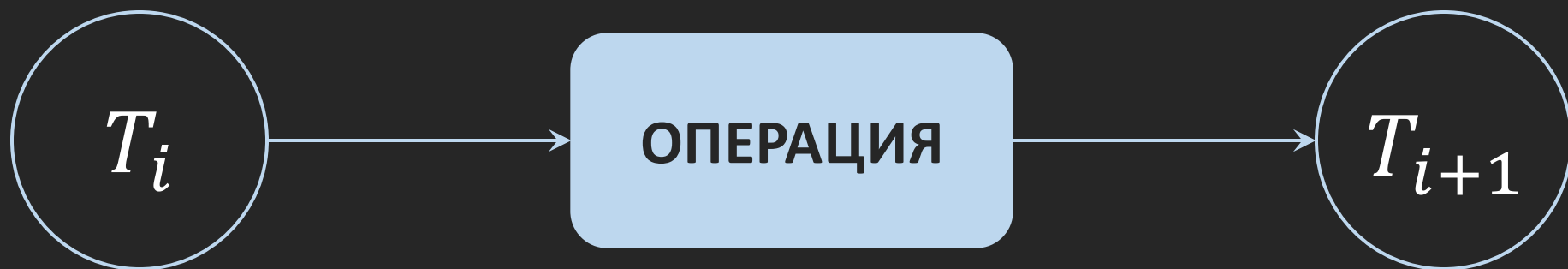
если вершина является правым потомком корня, то выполняем одинарный поворот относительно корня

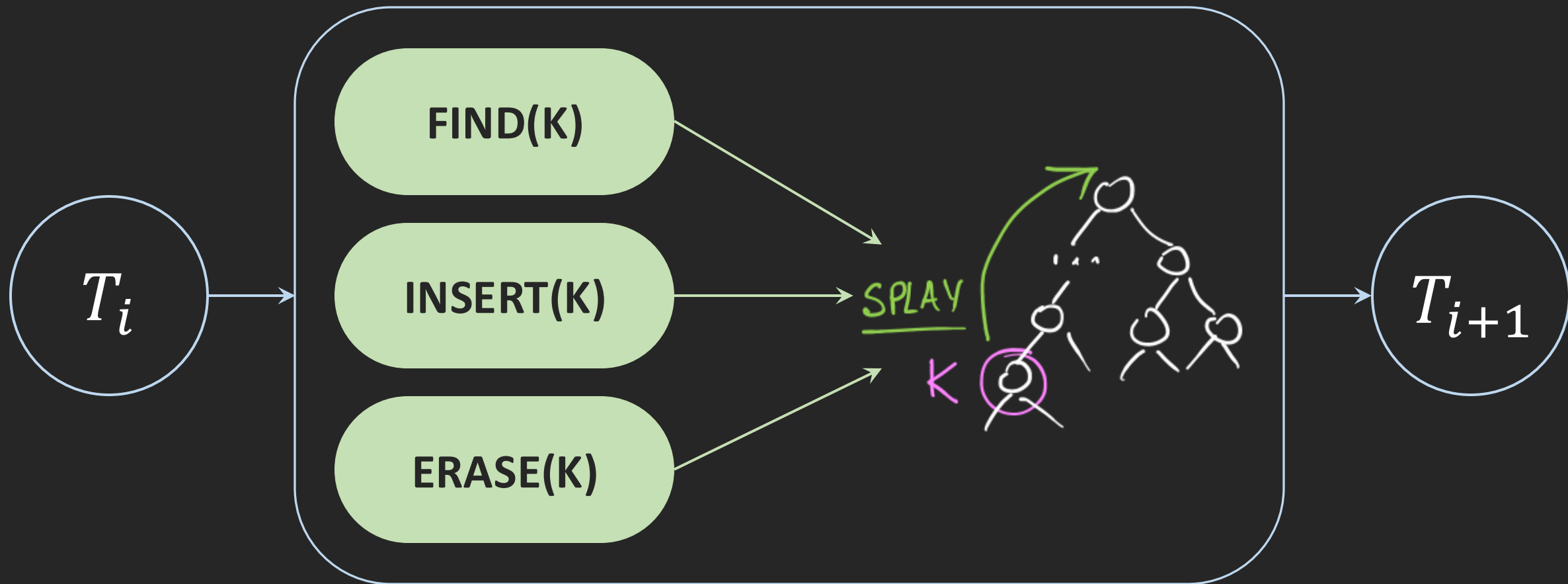


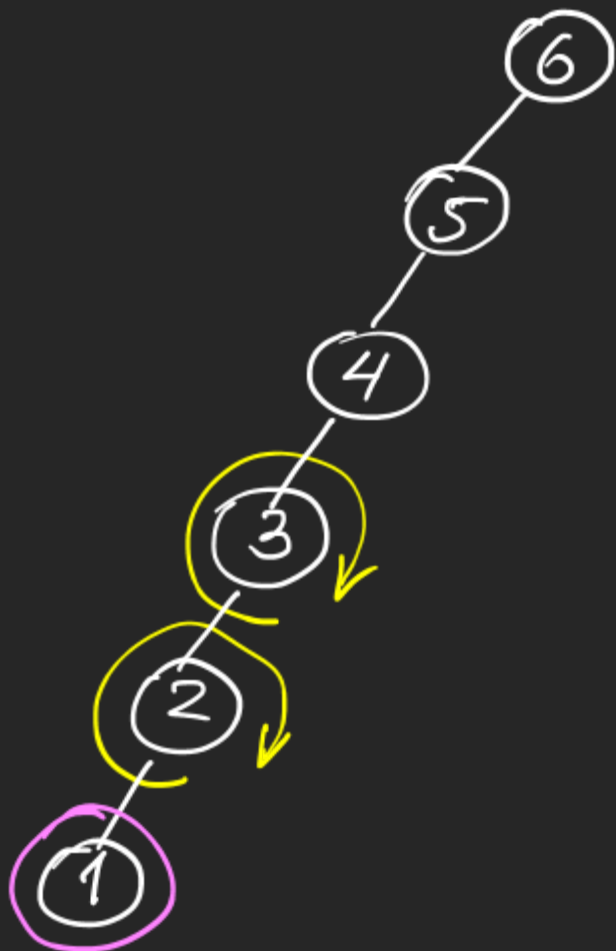
если вершина является правым [левый] потомком корня, то выполняем одинарный поворот относительно корня

операции **zig-zag**, **zig-zig** и **zig** работают с вершиной, которая **движется к корню**, а не с теми вершинами, относительно которых выполняются повороты

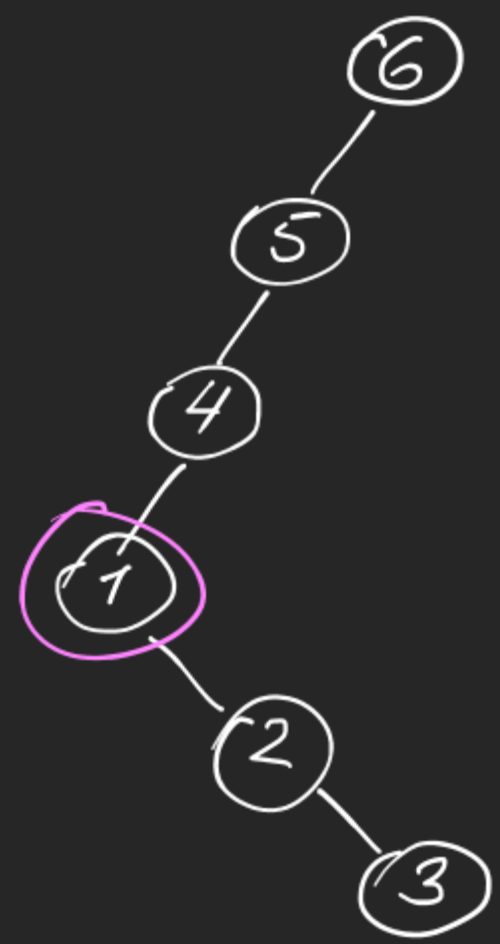
**СТОИТ
ОБРАТИТЬ
ВНИМАНИЕ**

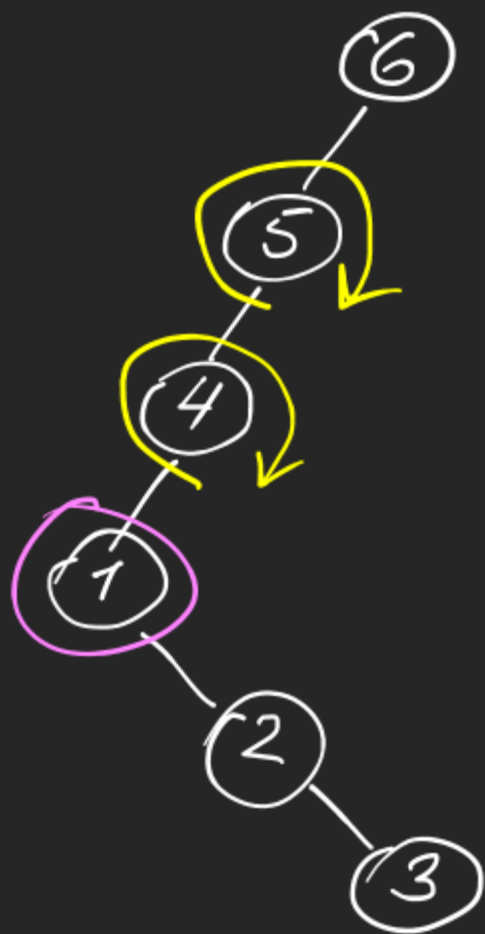




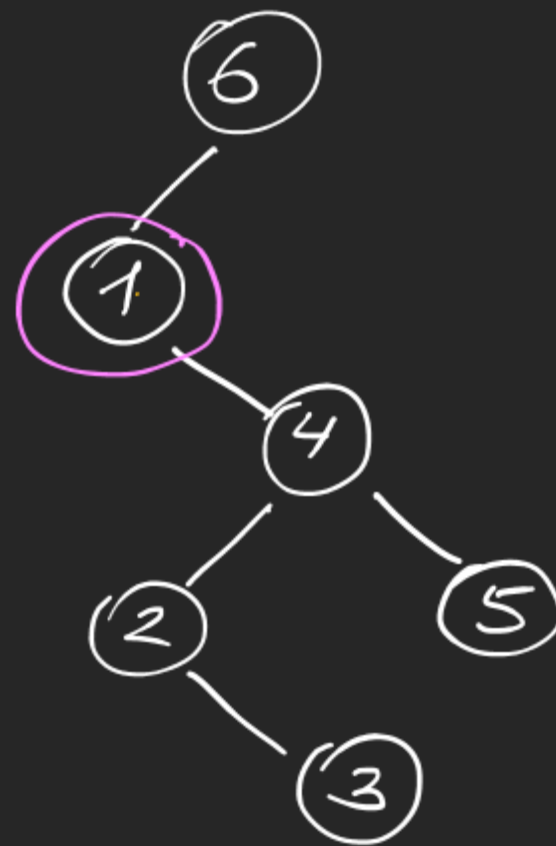


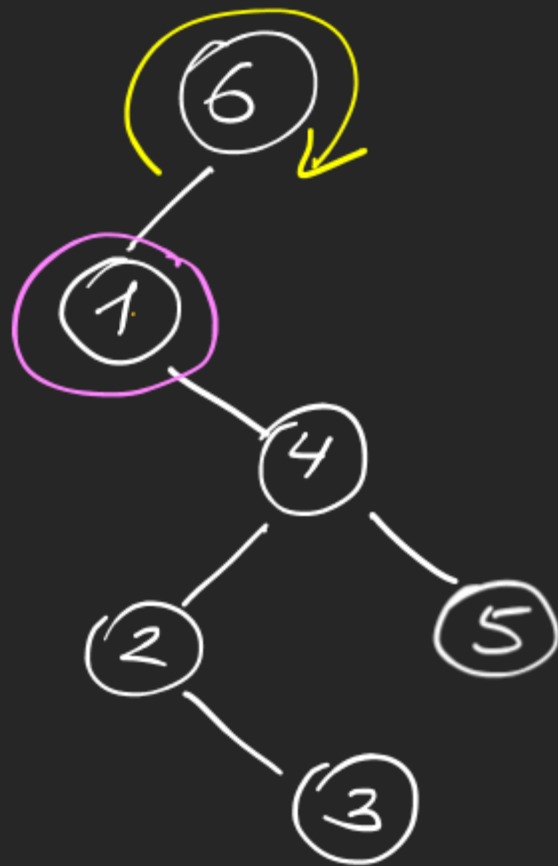
Zig-zig(1)





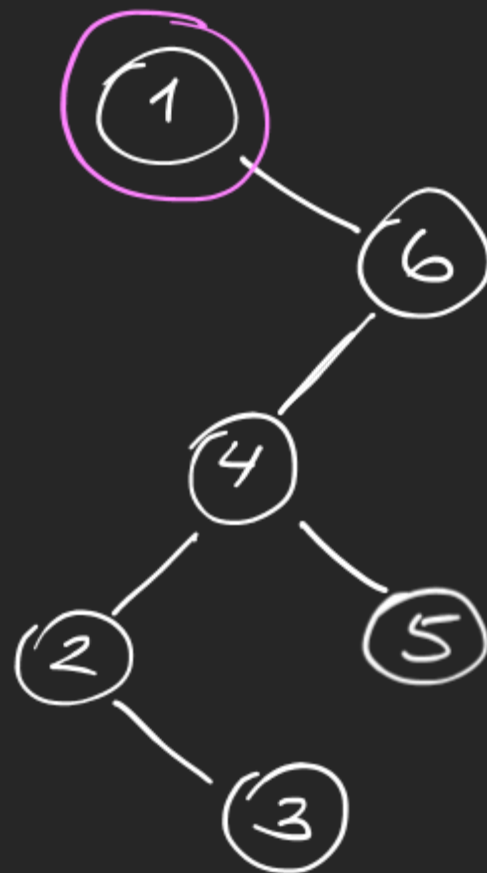
Zig-Zig(1)





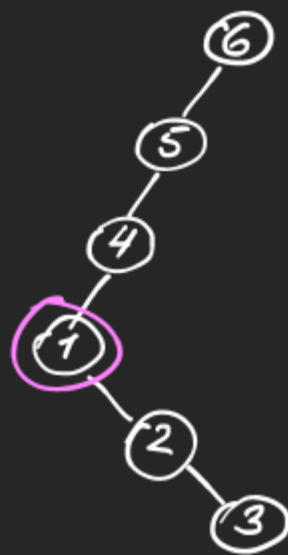
Zig(1)

→

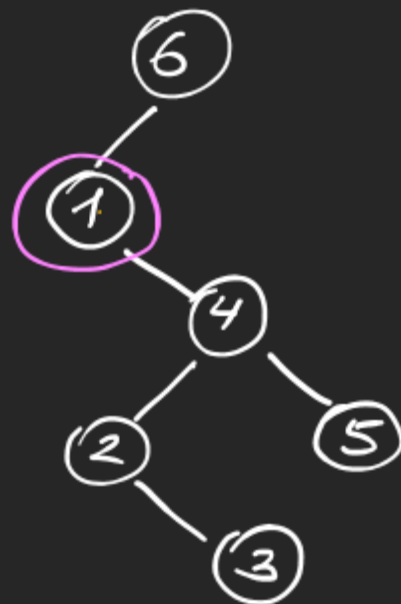




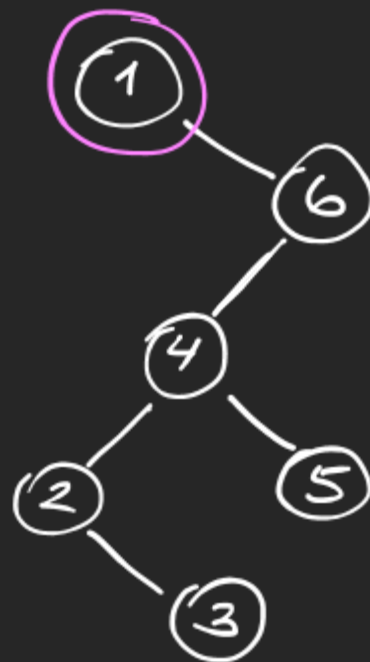
Zig-Zig



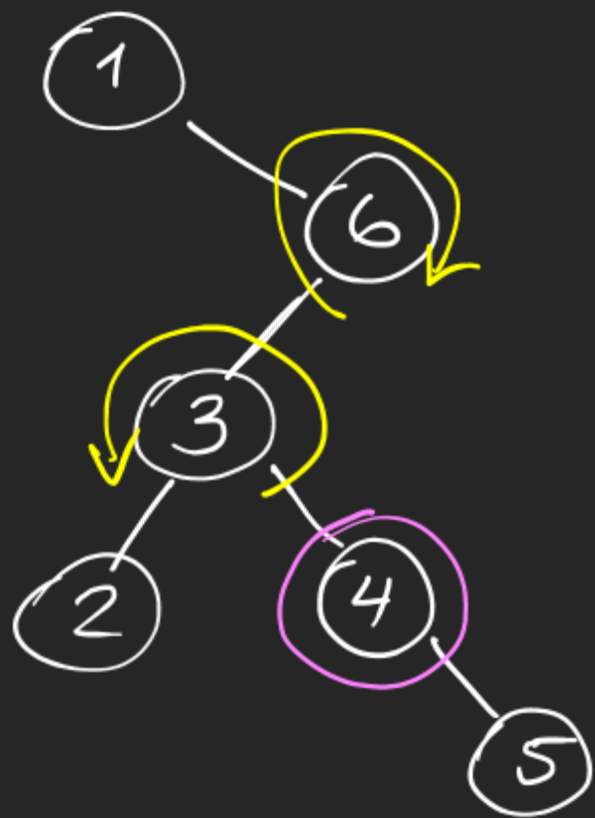
Zig-Zig



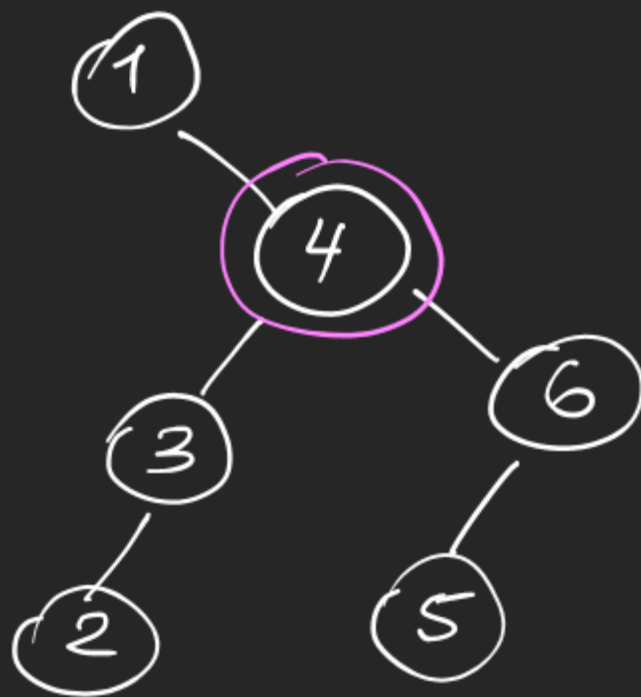
Zig

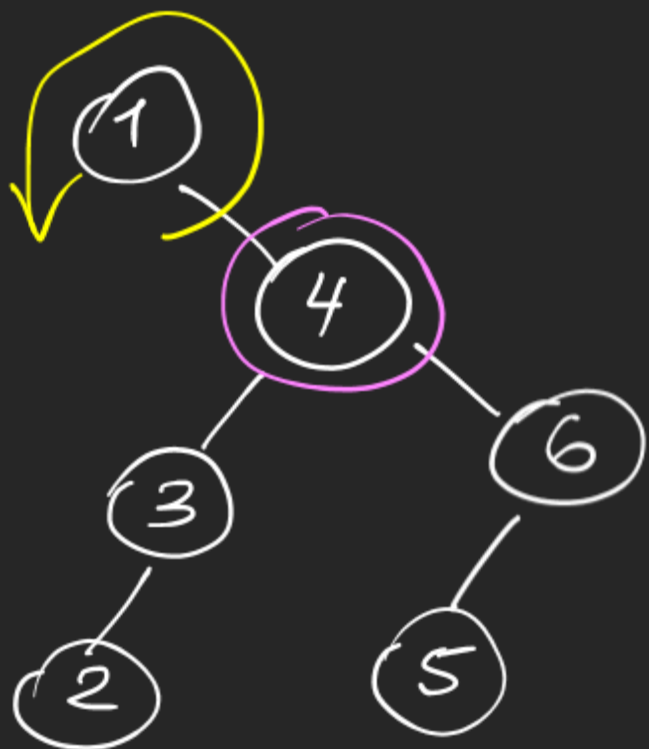


двойные повороты **zig—zig** и **zig—zag** дают
неплохой прогресс в плане **балансировки**
высоты получаемого дерева поиска

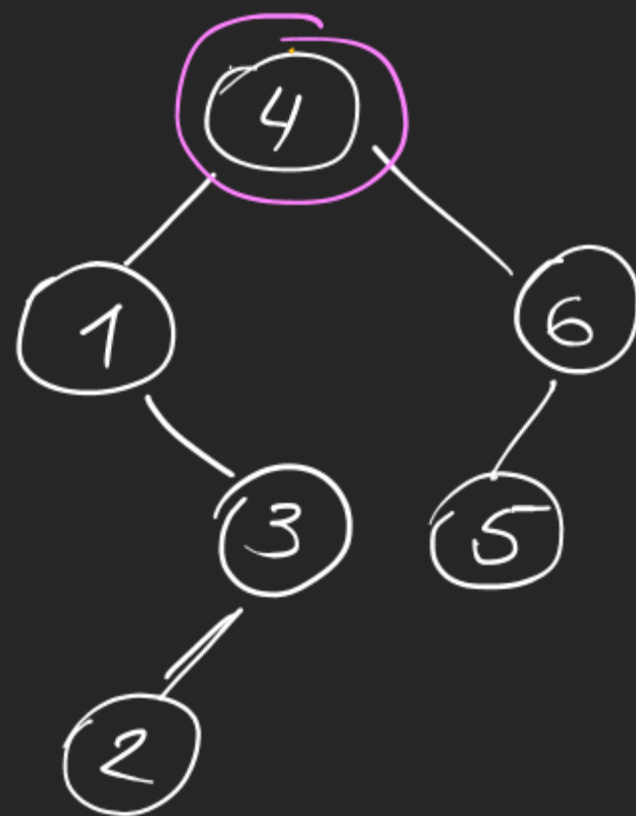


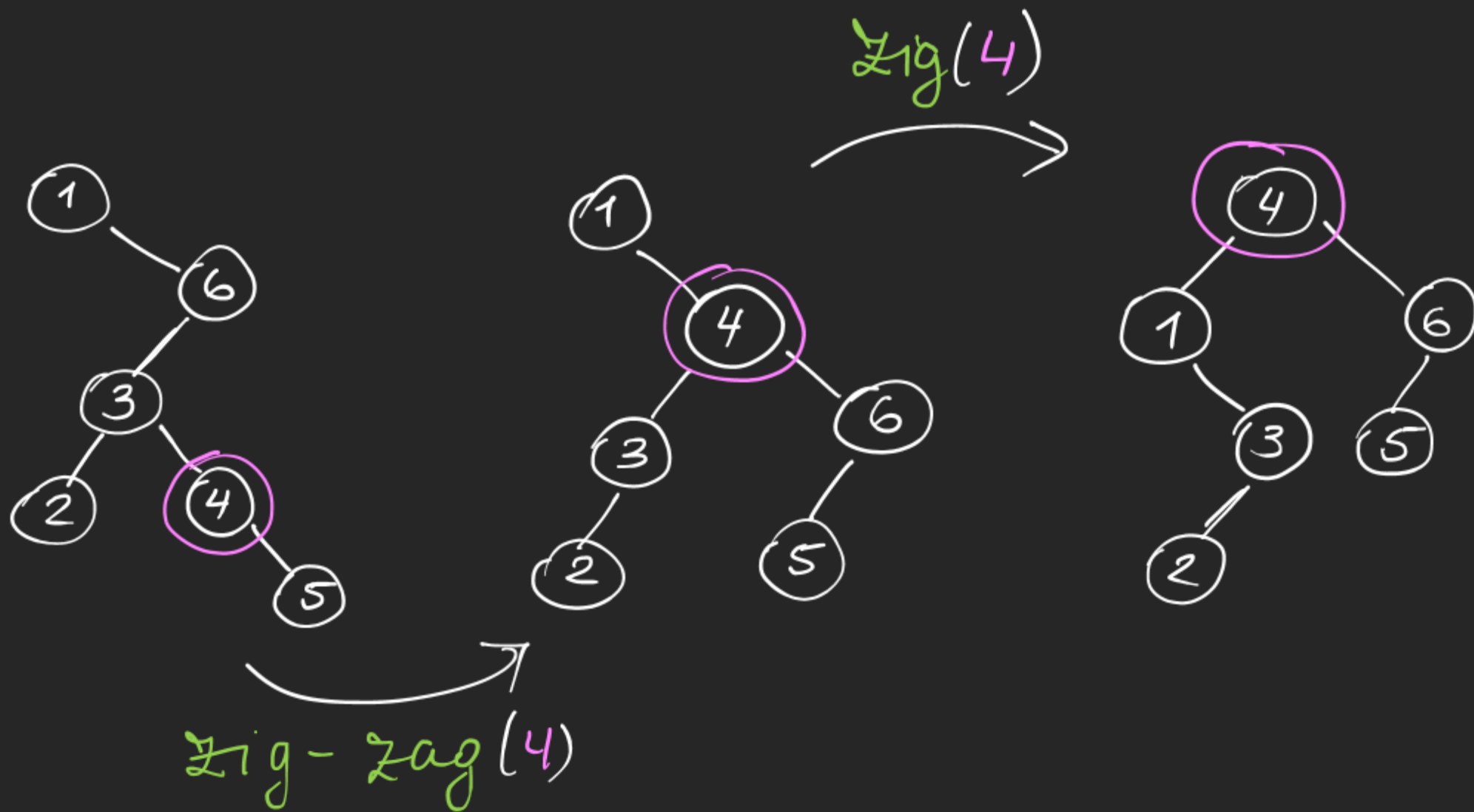
Zig-Zag (4)

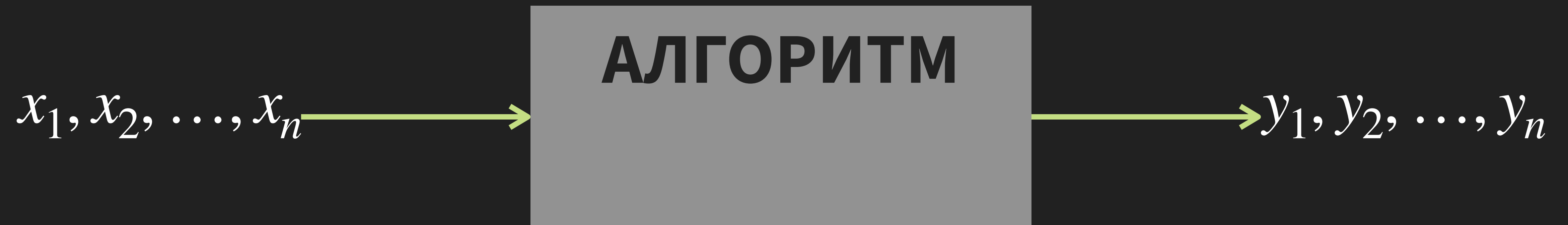


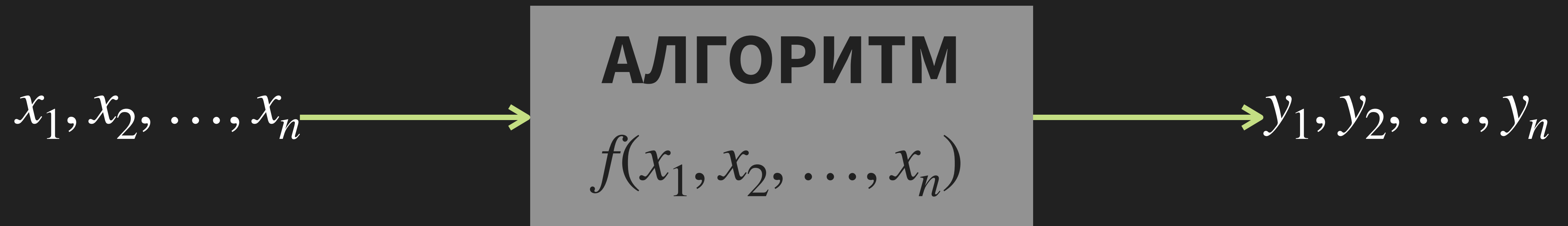


zig(4)



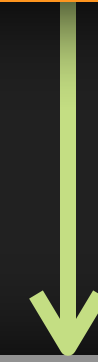






ДЕТЕРМИНИРОВАННЫЙ АЛГОРИТМ

случайная
величина



АЛГОРИТМ

$f(x_1, x_2, \dots, x_n, r)$

$x_1, x_2, \dots, x_n$

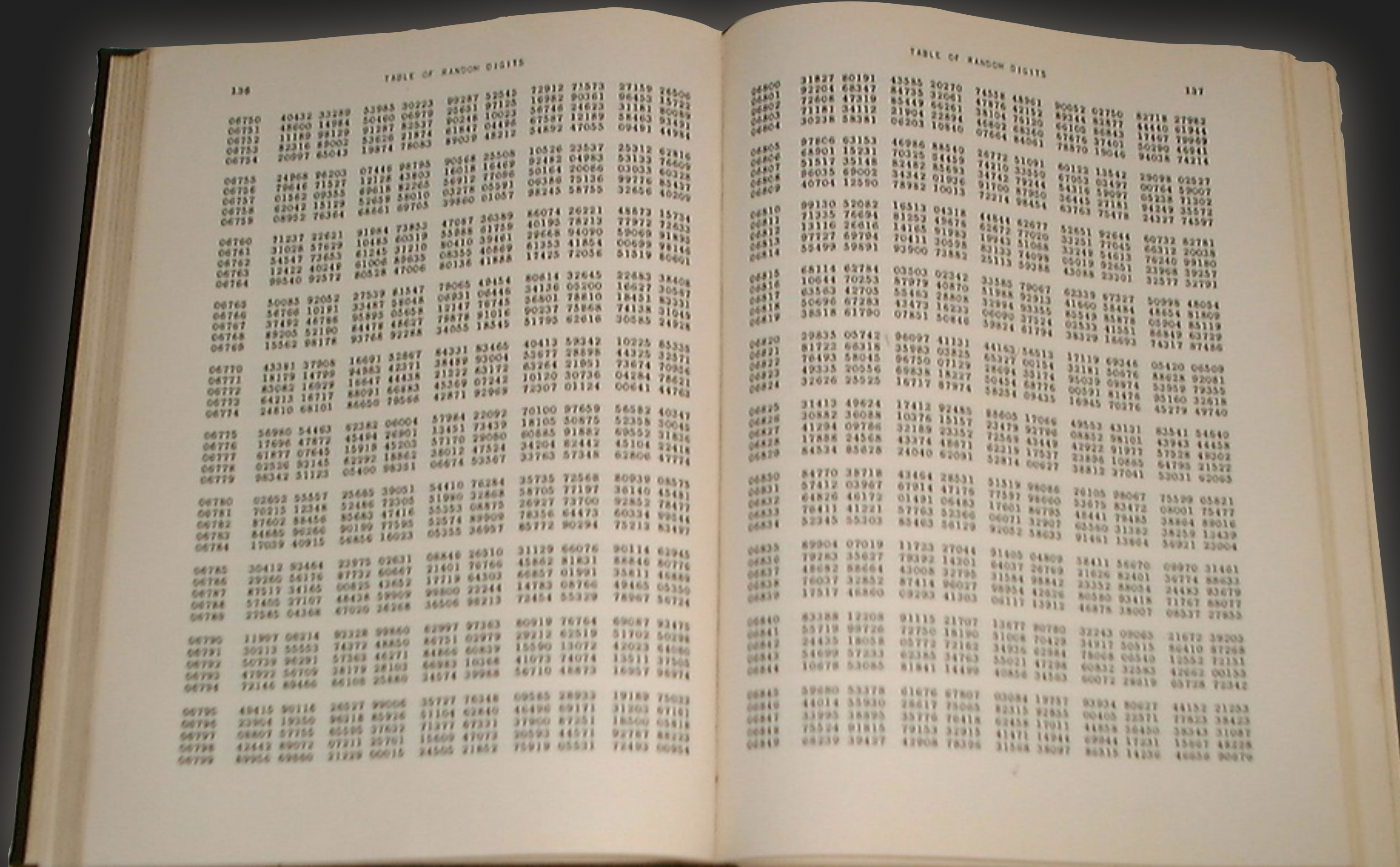


$y_1, y_2, \dots, y_n$

СТОХАСТИЧЕСКИЙ АЛГОРИТМ

О R O R O O O R R R O O O O R O R R R R R R R R O O O O O O O R R O

- 1 можно предположить, что эта последовательность **не является случайной**
- 2 какова **вероятность** того, что после 6 решек выпадет 6 орлов?



1955 A Million Random Digits with 100 000 Normal Deviates

генерация случайных чисел • стратегия 1

- ➔ использовать результаты некоторого эксперимента



- ➔ самый **неудобный** способ для реализации на компьютере

```
1  int getRandomNumber() {  
2      return 4;  
3  }
```

генерация случайных чисел • стратегия 2

```
1  auto time1 = chrono::high_resolution_clock::now();
2  cout << "enter a character:" << endl;
3  char blob;
4
5  cin >> blob;
6  auto time2 = chrono::high_resolution_clock::now();
7
8  auto time = chrono::duration_cast<chrono::milliseconds>(time2 - time1);
9  cout << time.count();
```

генерация случайных чисел • LCG

- ➔ порождение последовательности чисел, которые только **кажутся** случайными — ПСЕВДО-СЛУЧАЙНЫЕ ЧИСЛА
 - ➔ линейный конгруэнтный генератор
-

- (1) зафиксируем начальное значение x_0 — seed генератора
- (2) выберем два числа $A > 0$ и $M > 1$
- (3) следующее значение вычисляется рекурсивно $x_{i+1} = A \cdot x_i \bmod M$

M — простое число, а также A и x_0 — взаимно простые с M

$$x_{i+1} = A \cdot x_i \bmod M$$

→ при $M = 17$, $A = 6$ и $x_0 = 3$ получим

3 1 6 2 12 4 7 8 14 16 11 15 5 13 10 9 3 ...

→ изменение x_0 на 4, кажется, дает другой результат

4 7 8 14 16 11 15 5 13 10 9 3 1 6 2 12 4 ...

→ изменение A на 2 дает такой результат

3 6 12 7 14 11 5 10 3 6 12 7 14 11 5 10 3 ...

→ при $M = 2^{31} - 1$ и $A = 48721$ период составит $2^{31} - 2$



Всякий, кто использует
арифметические методы для
генерации случайных чисел,
разумеется, совершает грех...

Джон **фон НЕЙМАН** (1903–1957)

генерация случайных чисел • `std::rand()`

возвращает псевдо-случайное число $[0..RAND_MAX]$

- ➔ значение `RAND_MAX` ≥ 32767 определяется внутренней реализацией стандартной библиотеки
- ➔ другие алгоритмы в стандартной библиотеке могут использовать (а могут и нет 🤔)
- ➔ отсутствие гарантий потоковой безопасности

генерация случайных чисел • `std::mt19937()`

вихрь Мерсенна с периодом $2^{19937} - 1$

➔ конфигурирование рас
`std::uniform_int_di`
`std::normal_distrib`
`std::bernoulli_dist`
`std::poisson_distrib`
• • •

```
1  std::random_device rand_dev;  
2  std::mt19937 generator(rand_dev());  
3  std::uniform_int_distribution<> distr(0, 499);  
4  
5  for (int i = 0; i < 500; ++i) {  
6      a[i] = distr(generator);  
7  }
```

➔ средства стандартной библиотеки `#include<random>`

